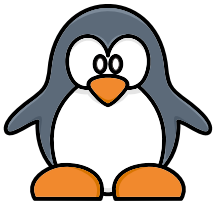
EMBEDDED LINUX
INTERVIEW PREPARATION ROADMAP

Complete Guide for Fresher & Junior Roles

Linux Kernel • Device Drivers • Build Systems • Debugging • Cross-Compilation • Real-Time







Yassin Omri

TABLE OF CONTENTS

1. Linux Fundamentals
2. Linux Kernel Architecture
3. Device Drivers Basics
4. Build Systems (Yocto, Buildroot)
5. Boot Process & Bootloaders
6. File Systems
7. Process & Thread Management
8. Inter-Process Communication (IPC)
9. Signals
10. Cross-Compilation & Toolchains
11. Hardware Protocols (I2C, SPI, UART)
12. Real-Time Linux
13. Debugging & Tracing Tools
14. Networking Basics
15. Shell Scripting & Automation
16. Common Interview Problems

1. LINUX FUNDAMENTALS

Essential Knowledge

Q1.1: Explain the Linux directory structure

Answer:

Linux organizes everything under a single root (/). Each directory has a specific purpose defined by the Filesystem Hierarchy Standard (FHS):

Directory	Purpose
/bin	Essential binaries (ls, cp, cat)
/dev	Device files — every hardware device appears here
/etc	System configuration files
/proc	Virtual filesystem exposing kernel/process info
/sys	Virtual filesystem for devices and drivers
/lib	Shared libraries and kernel modules
/tmp	Temporary files (often in RAM)
/var	Variable data: logs, caches, PID files

> **Tip: /proc and /sys are not real files on disk — the kernel generates them on the fly. Very useful for debugging.**

Q1.2: Explain Linux file permissions

Answer:

Every file has three permission groups — Owner, Group, and Others — each with read (r), write (w), and execute (x) bits. You can represent them as numbers: r=4, w=2, x=1.

In embedded systems, device file permissions under /dev matter a lot. If your app can't open /dev/i2c-1, it's usually a permissions issue.

```
$ ls -l myfile
-rwxr-xr-- 1 user group 1024 myfile
# rwx = owner can read, write, execute
# r-x = group can read and execute
# r-- = others can only read
$ chmod 755 myfile # rwxr-xr-x
$ chown user:group myfile
```

Q1.3: What is the difference between a hard link and a symbolic link?

Answer:

A hard link is another name pointing to the same file data (same inode). Deleting the original doesn't affect the hard link — the data stays as long as any link exists.

A symbolic link (symlink) is a shortcut that stores the path to the target. If the target is deleted, the symlink breaks. Symlinks can cross filesystem boundaries; hard links cannot.

Feature	Hard Link	Symbolic Link
Points to	Same file data (inode)	A file path
Cross filesystems	No	Yes
If original deleted	Still works	Broken link

Q1.4: What are stdin, stdout, stderr and how does I/O redirection work?

Answer:

Every Linux process has three default streams: stdin (input), stdout (normal output), stderr (error output). They are just file descriptors (0, 1, 2). Redirection works by pointing these descriptors to files or pipes before the program starts — the program doesn't know the difference.

```
$ ./app > output.txt # redirect stdout to file
$ ./app 2> errors.txt # redirect stderr to file
$ ./app > all.log 2>&1 # redirect both to same file
$ cat file | grep "text" # pipe stdout of cat into grep
```

Q1.5: What is the difference between a process and a thread?

Answer:

A process is an independent running program with its own memory space. Processes are isolated — a crash in one doesn't affect others.

A thread is an execution unit inside a process. All threads in the same process share memory, so communication is easy but requires synchronization to avoid data corruption.

Aspect	Process	Thread
Memory	Own private address space	Shared with all threads
Crash impact	Only that process affected	Can crash the whole process
Communication	Needs IPC (pipes, sockets)	Direct — shared memory
Creation cost	Heavier (fork)	Lighter (pthread_create)

2. LINUX KERNEL ARCHITECTURE

Q2.1: Describe the Linux kernel architecture

Answer:

Linux is a monolithic kernel — all core services (memory, scheduling, drivers, filesystems, networking) run together in kernel space. Despite this, it's modular: drivers and filesystems can be loaded/unloaded at runtime as kernel modules without rebooting.

The main subsystems are: Process Scheduler, Memory Manager, Virtual File System (VFS), Network Stack, and Device Drivers.

Q2.2: What is the difference between kernel space and user space?

Answer:

User space is where your applications run. It has limited access — it cannot directly touch hardware or other processes' memory. If it crashes, only that program is affected.

Kernel space is where the kernel runs. It has full access to hardware and all memory. A bug here crashes the whole system (kernel panic). Device drivers run in kernel space, which is why driver bugs are so serious.

Aspect	User Space	Kernel Space
Hardware access	Not allowed — must ask kernel	Direct access
Memory access	Own memory only	All system memory
Crash impact	Only that process	Entire system (kernel panic)
Examples	Your app, libraries	Kernel, device drivers

Q2.3: How do system calls work?

Answer:

When user space needs a kernel service (open a file, send data, allocate memory), it makes a system call. The CPU switches to kernel mode, the kernel performs the operation, then switches back to user mode with the result.

You never call system calls directly — the C library wraps them. For example, fopen() calls the open() system call underneath.

> **Tip: Use strace ./myprogram to see every system call your program makes. Invaluable when something fails and you don't know why.**

Q2.4: What are kernel modules?

Answer:

Kernel modules are pieces of kernel code (device drivers, filesystems) that can be loaded into a running kernel without rebooting. They have a .ko extension and are loaded with modprobe or insmod.

Every module has an init function (called on load) and an exit function (called on unload). The init function registers the driver with the kernel; the exit function must clean everything up.

```
$ lsmod # list loaded modules
$ modprobe usbhid # load a module
$ modprobe -r usbhid # unload a module
$ modinfo snd_hda_intel # show module info and parameters
```

Q2.5: How does the Linux scheduler work?

Answer:

The default scheduler is CFS (Completely Fair Scheduler). It tries to give every process a fair share of CPU time based on its priority (nice value). The process that has received the least CPU time recently runs next.

For real-time tasks, SCHED_FIFO and SCHED_RR policies exist with higher priority than normal tasks. Real-time tasks always preempt normal ones.

Policy	Type	Behavior
SCHED_NORMAL	CFS	Default — fair time sharing
SCHED_FIFO	Real-time	Runs until it blocks or yields — no time slice
SCHED_RR	Real-time	Like FIFO but with a time slice
SCHED_DEADLINE	Real-time	Task declares its own deadline

3. DEVICE DRIVERS BASICS

Q3.1: What are the three types of device drivers?

Answer:

Type	Interface	Examples
Character	Byte stream via /dev node	UART, SPI, I2C, GPIO, sensors
Block	Fixed-size blocks, random access	SD card, eMMC, SATA drives
Network	Packet-based via socket API	Ethernet, WiFi, CAN bus

Character drivers are most common in embedded work — almost every sensor or peripheral you write a driver for will be a character device.

Q3.2: What is the Linux driver model — how does a driver get matched to hardware?

Answer:

Linux uses a bus/driver/device model. A driver registers itself and declares which devices it supports (via a compatible string). When the kernel discovers a device — from the device tree or USB enumeration — it looks for a matching driver. When found, it calls the driver's probe() function.

probe() is where your driver does its work: gets hardware resources (memory addresses, IRQs), initializes the device, and registers it with a subsystem. remove() is the opposite — it must undo everything probe() did.

> **Tip: The most common driver bug for beginners: probe() succeeds but remove() doesn't free all resources. This causes memory leaks or crashes when the module is unloaded.**

Q3.3: What is ioctl and why does it exist?

Answer:

read() and write() only move data in and out. But hardware often needs configuration: set a baud rate, trigger a conversion, query a status. ioctl() is the mechanism for these device-specific control operations — it's basically an escape hatch for anything that doesn't fit read/write.

Each ioctl command has a unique number. User space calls ioctl(fd, COMMAND, &data;) and the driver handles it in its unlocked_ioctl function.

4. BUILD SYSTEMS (YOCTO & BUILDROOT)

Q4.1: What is an embedded Linux build system and why do you need one?

Answer:

An embedded Linux system is much more than just the kernel — you need a cross-compiler, bootloader, root filesystem, and all your libraries and applications, all built for the right target architecture. Doing this by hand is error-prone and nearly impossible to reproduce.

A build system automates the whole process from source to a bootable image, handling dependencies, versions, and cross-compilation automatically.

Q4.2: How does Yocto work?

Answer:

Yocto uses recipes (.bb files) to describe how to build each software package. Recipes are organized into layers (meta-*). The build engine is BitBake, which reads all recipes, resolves dependencies, and builds everything in the right order.

Yocto is powerful but has a steep learning curve. It's the right choice for commercial products that need to be maintained long-term across multiple boards.

Concept	What it is
Recipe (.bb file)	Instructions to fetch, build, and install one package
Layer (meta-*)	Collection of recipes — you stack layers for your product
BitBake	The build engine that executes all the recipes
Image recipe	Defines what goes into your final root filesystem

Q4.3: When would you choose Buildroot over Yocto?

Answer:

Buildroot is simpler and faster. It uses a menuconfig interface to select packages and builds everything with a single make command. A minimal system can be ready in under an hour.

Choose Buildroot for quick prototypes, simple appliances, or when the team is small and can't afford the Yocto learning curve. Choose Yocto for products with long lifespans, multiple hardware variants, or when you need field-updateable packages.

Factor	Yocto	Buildroot
Learning curve	Steep	Gentle
First build time	2–8 hours	15–60 minutes
Package management	Yes (opkg/rpm/deb)	No
Best for	Long-lived commercial products	Prototypes and simple systems

5. BOOT PROCESS & BOOTLOADERS

Q5.1: Walk through the embedded Linux boot sequence

Answer:

Stage	Who runs	What happens
1 — ROM code	SoC ROM	Fixed code in chip, loads next stage from flash/SD
2 — SPL	First-stage loader	Initializes DRAM, loads U-Boot
3 — U-Boot	Bootloader	Initializes peripherals, loads kernel + device tree into RAM, boots
4 — Linux Kernel	Kernel	Initializes drivers, mounts root filesystem, starts init
5 — Init/systemd	User space	Starts all system services

Q5.2: What is U-Boot and what role does it play?

Answer:

U-Boot is the most widely used bootloader for embedded Linux. It runs before the kernel and is responsible for hardware initialization the kernel can't do itself (DDR configuration, clocks), then loads the kernel image and device tree into RAM and hands control to the kernel.

U-Boot stores configuration in environment variables (saved to flash). This lets you change boot behavior — like the kernel location or boot arguments — without reflashing U-Boot itself.

```
# Common U-Boot commands
=> printenv # show all env variables
=> setenv bootargs 'console=ttyS0,115200 root=/dev/mmcblk0p2'
=> fatload mmc 0:1 0x80000000 zImage # load kernel from SD
=> bootz 0x80000000 - 0x81000000 # boot kernel
```

Q5.3: What is the Device Tree and why does it exist?

Answer:

Before device tree, every board needed its own hardcoded C code in the kernel to describe its hardware. This became unmanageable as the number of ARM boards exploded.

The device tree is a separate file (.dtb) that describes the hardware — what peripherals exist, their memory addresses, IRQ numbers, and how they connect. The kernel reads this file at boot and uses it to initialize drivers without any board-specific code. The bootloader (U-Boot) passes it to the kernel.

> **Tip: If a peripheral isn't working, check its status in the device tree first. status = 'disabled' means the driver won't probe it, even if the driver is loaded.**

6. FILE SYSTEMS

Q6.1: What filesystems are used in embedded Linux and why?

Answer:

The choice depends on the storage type and whether you need to survive power loss safely.

Filesystem	Storage	Writable	Use case
ext4	eMMC / SD card	Yes	General-purpose, journaled (safe on power loss)
SquashFS	Any	No (read-only)	Compressed read-only root filesystem
UBIFS	Raw NAND flash	Yes	Writable data on NAND flash
tmpfs	RAM	Yes	Temporary files — gone on reboot, very fast
NFS	Network	Yes	Development — avoids reflashing during dev

A common production pattern: SquashFS for the root filesystem (immutable, compressed) + a small ext4 partition for configuration and logs.

Q6.2: What is initramfs and why is it needed?

Answer:

initramfs is a tiny temporary root filesystem the kernel loads into RAM at boot. It contains just enough tools and drivers to mount the real root filesystem.

It solves a chicken-and-egg problem: to mount the real root filesystem you might need a driver, but that driver lives on the filesystem you're trying to mount. initramfs provides that driver first, then hands off to the real root.

In embedded, initramfs is sometimes used as the permanent root filesystem for security-sensitive or very minimal devices.

7. PROCESS & THREAD MANAGEMENT

Q7.1: How does process creation work in Linux?

Answer:

fork() creates a child process by duplicating the parent. The child gets a copy of the parent's memory (using copy-on-write — actual copying only happens if either side writes). The child then usually calls exec() to replace itself with a different program.

The parent should call wait() after forking to collect the child's exit status. If it doesn't, the child becomes a zombie process — it's exited but still occupies a slot in the process table.

```
pid_t pid = fork();
if (pid == 0) {
execl("/bin/ls", "ls", "-l", NULL); // child runs ls
} else {
wait(NULL); // parent waits for child to finish
}
```

Q7.2: What is a mutex and why do you need it?

Answer:

When two threads share data, a race condition can occur: both read the same value, modify it, and write back — one update gets lost. The result is unpredictable and hard to reproduce.

A mutex (mutual exclusion lock) ensures only one thread can access the shared data at a time. The other thread waits until the first is done. Keep the critical section (the locked region) as short as possible.

```
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_lock(&lock);
counter++; // only one thread here at a time
pthread_mutex_unlock(&lock);
```

> **Tip: Deadlock: thread A waits for lock B while holding lock A; thread B waits for lock A while holding lock B. Prevention: always acquire multiple locks in the same order everywhere.**

Q7.3: What is the difference between a mutex and a semaphore?

Answer:

A mutex has an owner — only the thread that locked it can unlock it. It's for protecting a critical section.

A semaphore is a counter with no owner. It's for signaling between threads — a producer increments it when data is ready, a consumer decrements it (blocking if zero). A binary semaphore looks like a mutex but is meant for synchronization, not mutual exclusion.

8. INTER-PROCESS COMMUNICATION (IPC)

Q8.1: Compare the main IPC mechanisms

Answer:

Mechanism	Data model	Best for
Pipe	Byte stream, one direction	Simple parent-child data flow
Named pipe (FIFO)	Byte stream	Unrelated processes on same machine
Message queue	Discrete messages with priority	Structured command dispatch
Shared memory	Raw memory — no copy	High-bandwidth data, fastest IPC
Unix socket	Stream or datagram, bidirectional	Flexible local IPC

Shared memory is the fastest because no data is copied — both processes see the same physical memory. The tradeoff: you must synchronize access yourself with a mutex or semaphore.

Q8.2: How does a pipe work internally?

Answer:

A pipe is a kernel buffer with two ends: a write end and a read end. Writing to the pipe copies data into the kernel buffer. Reading takes it out. If the buffer is full, the writer blocks. If it's empty, the reader blocks. No data ever touches disk.

When the write end is closed and the buffer is empty, read() returns 0 (EOF) — this is how shell pipelines know a command has finished.

```
int pipefd[2];
pipe(pipefd); // pipefd[0]=read end, pipefd[1]=write end
if (fork() == 0) {
close(pipefd[1]); // child only reads
read(pipefd[0], buf, sizeof(buf));
} else {
close(pipefd[0]); // parent only writes
write(pipefd[1], "hello", 5);
}
```

9. SIGNALS

NEW SECTION

Q9.1: What are signals and how are they used?

Answer:

A signal is an asynchronous notification sent to a process — a simple way for the kernel or another process to say 'something happened'. The process can handle it with a custom function, ignore it, or let the default action happen (usually terminate).

Signal	Default	Common use
SIGTERM (15)	Terminate	Ask a process to shut down gracefully
SIGKILL (9)	Terminate (uncatchable)	Force kill — cannot be caught or ignored
SIGINT (2)	Terminate	Sent when user presses Ctrl+C
SIGHUP (1)	Terminate	Daemons use it to reload configuration
SIGUSR1/2	Terminate	Application-defined custom events
SIGSEGV (11)	Core dump	Invalid memory access

Q9.2: Why use sigaction instead of signal()?

Answer:

signal() is old and its behavior varies between systems — on some it resets the handler after each signal, causing bugs. sigaction() is the POSIX standard: predictable, portable, and gives you control over edge cases like what to do when the signal arrives while the handler is already running.

Best practice: keep signal handlers tiny — just set a flag. Do the real work in the main loop where you have full context.

```
volatile sig_atomic_t g_stop = 0;
void handler(int sig) { g_stop = 1; } // just set a flag
struct sigaction sa = { .sa_handler = handler, .sa_flags = SA_RESTART };
sigemptyset(&sa.sa_mask);
sigaction(SIGTERM, &sa, NULL);
while (!g_stop) { /* do work */ }
/* clean up here */
```

10. CROSS-COMPILATION & TOOLCHAINS

NEW SECTION

Q10.1: What is cross-compilation and why is it needed?

Answer:

Cross-compilation means building code on your development machine (x86 PC) that will run on a different architecture (ARM, RISC-V). It's needed because embedded targets are often too slow or too small to compile large codebases themselves.

A cross-toolchain is a special compiler that runs on your host but produces binaries for the target. You select it by setting the CROSS_COMPILE environment variable.

```
export CROSS_COMPILE=arm-linux-gnueabihf-
export ARCH=arm
${CROSS_COMPILE}gcc hello.c -o hello # builds for ARM
file hello # confirm: ELF 32-bit, ARM — not x86!
```

Tip: Always run 'file ./binary' after cross-compiling. If it says x86-64, you accidentally built it natively.

Q10.2: What is a sysroot?

Answer:

A sysroot is a directory that mirrors the target's filesystem, containing the target's headers and libraries. The cross-compiler uses it to link against the correct target libraries instead of your host's x86 libraries.

Without a correct sysroot, you'll either get linker errors or silently link against the wrong libraries. Build systems like Yocto generate an SDK that packages the sysroot and toolchain together.

Q10.3: What are common ARM toolchain triplets?

Answer:

Triplet	For
arm-linux-gnueabi-	32-bit ARM, soft-float (older hardware)
arm-linux-gnueabihf-	32-bit ARM, hard-float (ARMv7+, most common)
aarch64-linux-gnu-	64-bit ARM (modern SoCs)
riscv64-linux-gnu-	64-bit RISC-V

11. HARDWARE PROTOCOLS: I2C, SPI, UART

NEW SECTION

Q11.1: Explain I2C — how it works and when to use it

Answer:

I2C uses just two wires (SDA for data, SCL for clock) and supports multiple devices on the same bus. Each device has a unique address. The master always initiates communication. It's slow (100kHz–400kHz typically) but very pin-efficient.

Best for: sensors, EEPROMs, displays, power management ICs — anything low-speed where pin count matters. From Linux, devices appear either as /dev/i2c-N (accessed directly) or as higher-level devices under /sys.

```
$ i2cdetect -y 1 # scan bus 1 for devices
$ i2cget -y 1 0x48 0x00 # read register 0x00 from device 0x48
```

Q11.2: Explain SPI — how it differs from I2C

Answer:

SPI uses four wires: MOSI (data to slave), MISO (data from slave), SCLK (clock), and CS (chip select — one per device). It's full-duplex and much faster than I2C (can reach 50MHz+).

Best for: flash memory, display controllers, ADCs, high-speed sensors. The tradeoff: each device needs its own CS line, so adding many devices costs GPIO pins.

Property	I2C	SPI
Wires	2 + power	4 + 1 CS per device
Speed	100kHz – 3.4MHz	1MHz – 100MHz+
Direction	Half-duplex	Full-duplex (simultaneous TX/RX)
Best for	Many slow devices	Fast peripherals: flash, displays

Q11.3: What is UART and what is it used for?

Answer:

UART is the simplest serial protocol — just TX and RX wires, no clock. Both sides must agree on the baud rate in advance. It's asynchronous: each byte is self-framed with start/stop bits.

In embedded Linux, the debug console is almost always a UART (/dev/ttyS0 or /dev/ttyAMA0). It's your first window into a new board — when nothing else works, the UART console tells you what's happening. Also used for GPS modules, Bluetooth serial, and GSM modems.

> **Tip: If UART output looks like garbage symbols, the baud rate on both sides doesn't match. That's the first thing to check.**

12. REAL-TIME LINUX

NEW SECTION

Q12.1: What does 'real-time' mean and how does it differ from 'fast'?

Answer:

Real-time means deterministic — the system guarantees a task completes within a defined deadline, every single time. It doesn't mean fast; it means predictable.

A media player that occasionally drops a frame is fast but not real-time. A motor controller that must execute its control loop every 1ms with no exceptions is a real-time requirement — missing it could cause hardware damage.

Type	Deadline miss consequence	Examples
Hard real-time	System failure — unacceptable	Motor control, medical devices
Soft real-time	Degraded quality — tolerable	Audio/video streaming

Q12.2: What is PREEMPT_RT and why is it needed?

Answer:

Standard Linux has regions where even a high-priority task must wait — inside interrupt handlers or spinlocks, nothing else can run. This can cause latency spikes of several milliseconds, which is too slow for real-time tasks.

The PREEMPT_RT kernel patch makes Linux fully preemptible: high-priority tasks can interrupt almost anything. This reduces worst-case latency from tens of milliseconds to under 100 microseconds. You can measure it with the cyclictest tool.

```
# Measure worst-case latency
$ cyclictest -t 4 -p 80 -i 1000 -l 10000
# Min: 8us Avg: 12us Max: 45us <- aim for Max < 100us
```

Q12.3: What is priority inversion and how is it solved?

Answer:

Priority inversion: a high-priority task (H) is blocked waiting for a mutex held by a low-priority task (L). A medium-priority task (M) preempts L because it has higher priority than L. Now H is effectively blocked by M — the opposite of what priorities should do.

The fix is priority inheritance: when H is blocked on L's mutex, L temporarily gets H's priority so M can't preempt it. This is enabled by setting PTHREAD_PRIO_INHERIT on the mutex attribute.

> **Tip: The Mars Pathfinder rover hit this exact bug in 1997 and reset repeatedly until engineers patched it remotely. Priority inheritance was already available but not enabled.**

13. DEBUGGING & TRACING TOOLS

Q13.1: What tools do you use to debug embedded Linux issues?

Answer:

Tool	What it does	When to use
gdb / gdbserver	Step through code, inspect variables	Application crash or wrong behavior
strace	Shows every system call with result	Unexpected errors, permission issues
valgrind	Finds memory leaks and corruption	Memory bugs in applications
perf	Shows CPU hotspots	High CPU usage, performance tuning
ftrace / trace-cmd	Traces kernel function calls	Kernel and driver issues, RT latency
dmesg	Kernel log messages	Driver init errors, hardware issues

Q13.2: How do you debug a crash on a remote embedded target?

Answer:

Run gdbserver on the target and connect to it with gdb on your host. You can set breakpoints, step through code, and inspect variables exactly as if the program were running locally — you just need the unstripped binary (with debug symbols) on your host.

```
# On the target
$ gdbserver :1234 ./my_app
# On the host
$ arm-linux-gnueabihf-gdb ./my_app
(gdb) target remote 192.168.1.100:1234
(gdb) break main
(gdb) continue
```

Q13.3: How do you find a memory leak?

Answer:

A memory leak is heap memory that gets allocated but is never freed, and the pointer to it is lost. Over time, the process grows until it's killed by the OS.

valgrind --leak-check=full runs your program and reports every unfreed allocation with its call stack. For faster detection during development, compile with -fsanitize=address (AddressSanitizer) — it also catches buffer overflows and use-after-free bugs at only ~2x slowdown.

On a live embedded system, watch the VmRSS field in /proc/PID/status — if it keeps growing, you have a leak.

14. NETWORKING BASICS

Q14.1: How is networking configured in embedded Linux?

Answer:

Use the ip command (iproute2) to configure interfaces. In production, a daemon like systemd-networkd or connman handles configuration automatically at boot.

```
$ ip addr show eth0 # show interface and IP
$ ip link set eth0 up # bring interface up
$ ip route add default via 192.168.1.1 # set gateway
$ ss -tulpn # show open ports
```

> **Tip: On a new board, if networking doesn't work, check dmesg for PHY errors. The Ethernet PHY often needs a specific driver and correct device tree binding.**

Q14.2: What is the difference between TCP and UDP?

Answer:

TCP is reliable and ordered — the kernel handles retransmission, ordering, and flow control automatically. Use it when you can't afford to lose data: configuration, file transfer, MQTT.

UDP is unreliable but fast — packets can be lost or arrive out of order, and that's your problem. Use it when timeliness matters more than reliability: sensor streaming, real-time video, NTP time sync.

Q14.3: Write a basic TCP client/server

Answer:

```
// Server — listen for one connection
int srv = socket(AF_INET, SOCK_STREAM, 0);
struct sockaddr_in addr = {.sin_family=AF_INET,
.sin_port=htons(8080), .sin_addr.s_addr=INADDR_ANY};
bind(srv, (struct sockaddr*)&addr, sizeof(addr));
listen(srv, 3);
int cli = accept(srv, NULL, NULL); // blocks until client connects
char buf[256]; read(cli, buf, sizeof(buf));
send(cli, "OK", 2, 0);
close(cli); close(srv);
// Client — connect and send
int s = socket(AF_INET, SOCK_STREAM, 0);
struct sockaddr_in sa = {.sin_family=AF_INET, .sin_port=htons(8080)};
inet_pton(AF_INET, "127.0.0.1", &sa.sin_addr);
connect(s, (struct sockaddr*)&sa, sizeof(sa));
send(s, "Hello", 5, 0);
```

15. SHELL SCRIPTING & AUTOMATION

Q15.1: What are the most important shell scripting practices?

Answer:

Start every script with set -euo pipefail: exit on error, error on unset variables, and catch failures in pipes. These three flags prevent the most common silent bug patterns.

Always quote variables ("$var" not $var) to handle spaces in values. Check that required tools exist before using them. Use meaningful variable names and add comments for anything non-obvious.

```
#!/bin/bash
set -euo pipefail
TARGET="${1:?Error: target IP required}"
BINARY="./build/myapp"
[ -f "$BINARY" ] || { echo "Build first!"; exit 1; }
scp "$BINARY" "root@$TARGET:/opt/myapp"
ssh "root@$TARGET" "systemctl restart myapp"
```

Q15.2: Write a deployment script for an embedded target

Answer:

```
#!/bin/bash
set -euo pipefail
TARGET="${1:?Usage: deploy.sh <ip>}"
log() { echo "[$(date +%H:%M:%S)] $*"; }
# Build
log "Building..."
make -j"$(nproc)" CROSS_COMPILE=arm-linux-gnueabihf-
# Deploy
log "Copying to $TARGET..."
scp build/myapp root@"$TARGET":/opt/myapp/
# Restart service
log "Restarting..."
ssh root@"$TARGET" "systemctl restart myapp && systemctl status myapp"
log "Done!"
```

16. COMMON INTERVIEW PROBLEMS

This section is intentionally code-focused — these are the hands-on coding and debugging problems interviewers ask you to solve.

Problem 1: Find and fix memory leaks

Answer:

Spot the pattern: any early return that skips free(). The clean fix is a single exit point using goto — an accepted C idiom for error paths.

```
// BUGGY: leaks buffer when read fails
void process(int fd) {
char *buf = malloc(1024);
if (read(fd, buf, 1024) < 0) return; // LEAK!
use(buf);
free(buf);
}
// FIXED: single exit point
void process(int fd) {
char *buf = malloc(1024);
if (!buf) return;
if (read(fd, buf, 1024) < 0) goto done;
use(buf);
done:
free(buf); // always reached
}
```

Problem 2: Debug a segmentation fault

Answer:

Enable core dumps, reproduce the crash, open the core in GDB. The backtrace shows exactly where it crashed; inspecting the variables in that frame reveals the bad pointer.

```
$ ulimit -c unlimited && ./program
Segmentation fault (core dumped)
$ gdb ./program core
(gdb) backtrace # find the crash location
(gdb) frame 1 # examine the calling frame
(gdb) print item # check — is it NULL?
```

Common causes: NULL pointer not checked after malloc/lookup, array out of bounds (off-by-one), use-after-free, uninitialized pointer.

Problem 3: Fix busy-wait / high CPU usage

Answer:

A polling loop that spins checking a condition burns 100% CPU. Replace it with a blocking call that lets the kernel wake your thread when data is ready.

```
// BAD: spinning at 100% CPU
while (!(reg->status & READY)) { }
// GOOD: use poll() — blocks until data is available, zero CPU waste
struct pollfd pfd = { .fd = dev_fd, .events = POLLIN };
poll(&pfd, 1, -1);
// If you must poll (no interrupt support), at least yield:
while (!(reg->status & READY)) usleep(100);
```

Problem 4: Implement a ring buffer

Answer:

A ring buffer is a fixed-size circular queue — used in UART drivers, DMA buffers, and any producer/consumer scenario where you want minimal overhead.

```
#define SIZE 64
typedef struct { uint8_t buf[SIZE]; int head, tail; } RingBuf;
int rb_push(RingBuf *rb, uint8_t val) {
int next = (rb->head + 1) % SIZE;
if (next == rb->tail) return -1; // full
rb->buf[rb->head] = val;
rb->head = next;
return 0;
}
int rb_pop(RingBuf *rb, uint8_t *val) {
if (rb->head == rb->tail) return -1; // empty
*val = rb->buf[rb->tail];
rb->tail = (rb->tail + 1) % SIZE;
return 0;
}
```

INTERVIEW TIPS & RESOURCES

Technical Preparation

- Get a Raspberry Pi or BeagleBone and connect a real I2C sensor — read actual data
- Write a simple kernel module, load it, and read from it via /dev
- Build a complete image with Buildroot — understand every file in output/images/
- Cross-compile an application and confirm with 'file' that it's ARM
- Reproduce a segfault intentionally, then debug it with GDB from scratch
- Trace system calls of a failing program with strace — find the exact error

How to Stand Out

- Explain the 'why' — interviewers want reasoning, not just memorized answers
- Be honest about what you don't know — then explain how you'd find out
- Mention real projects, even small ones: a Raspberry Pi sensor project shows more than theory
- Ask about the team's hardware and challenges — shows genuine interest

Key Resources

- Linux Device Drivers 3rd Ed. (LDD3) — free at lwn.net — best book for beginners
- Bootlin training slides — bootlin.com/training — free, professional quality
- The Linux Programming Interface (TLPI) — Michael Kerrisk — definitive reference
- Embedded Linux Wiki — elinux.org — practical board-specific knowledge
- Yocto Mega-Manual — docs.yoctoproject.org

Good luck with your interviews!

Connect on LinkedIn: Yassin Omri